# پیاده‌سازی و ارزیابی Routing Methodologies روی RouterBench

> از یازده methodology سند Advanced LLM Routing Methodologies، سه مورد که پیش‌نیاز دادهٔ آن‌ها با RouterBench برآورده می‌شد پیاده‌سازی و روی ۷٬۲۹۶ پرامپت held-out ارزیابی شدند. یکی از آن‌ها (Intent-Driven Anchoring) به بهبود معنادار روی Pareto frontier رسید: هم‌زمان کیفیت بالاتر و هزینهٔ ۱۶٪ کمتر نسبت به بهترین سیاست بدون routing. در کنار آن، TTFT و TPOT برای دو مدل قابل host روی GPU خودمان اندازه‌گیری شد تا بُعد سوم utility function از حالت خالی در بیاید.

## ۱. نقطهٔ شروع و مسئله‌ای که باید حل می‌شد

یک baseline از قبل موجود بود: یک LLM به‌عنوان judge برای هر prompt تصمیم می‌گرفت کدام مدل باید پاسخ دهد. این baseline روی ۱۰۰ نمونهٔ تصادفی اجرا شده و عدد ۶۱٫۲۵٪ کیفیت با هزینهٔ ۰٫۰۰۰۲۲۳ دلار به‌ازای هر query گزارش کرده بود.

مشکل این عدد کیفیتش نبود، تفسیرپذیری‌اش بود. یک نتیجهٔ routing تنها در مقایسه با سه مرجع معنا پیدا می‌کند و هیچ‌کدام محاسبه نشده بودند:

- سقف (oracle) — بالاترین کیفیتی که با انتخاب بهترین مدل برای هر prompt اصلاً قابل دسترسی است
- کف (best constant) — نتیجهٔ حالتی که اصلاً routing نکنیم و همیشه یک مدل ثابت را صدا بزنیم
- مرجع تصادفی (random) — آنچه شانس محض می‌دهد

وقتی این سه را محاسبه کردیم، تصویر عوض شد. روی همان ۱۰۰ پرامپت، سیاست ثابت always Yi-34B به ۶۵٫۰۰٪ با هزینهٔ ۰٫۰۰۰۱۷۶ می‌رسید — یعنی هم کیفیت بالاتر و هم هزینهٔ کمتر از judge. توزیع تصمیم‌ها هم گویا بود: judge در ۹۱ مورد از ۱۰۰ مورد یک مدل واحد را انتخاب کرده بود، و آن ۹ باری که منحرف شد روی همان سطرها ۲۲٫۲٪ گرفت در حالی که انتخاب ثابت ۳۳٫۳٪ می‌داد.

> نکته‌ای که باید در گزارش نهایی بیاید: در n=۱۰۰ فاصلهٔ اطمینان این تفاوت‌ها صفر را در بر می‌گیرد (paired bootstrap: −۳٫۷۵ واحد درصد با CI95 برابر [−۱۳٫۷۵, +۶٫۲۵]). ادعای درست این است که آزمایش نتوانست برتری judge را نشان دهد، نه اینکه بدتر بودنش اثبات شد. تفکیک این دو در داوری اهمیت دارد.

دو ایراد در مسیر داده هم پیدا و رفع شد که مستقل از انتخاب methodology روی نتیجه اثر می‌گذاشتند: ستون prompt در RouterBench آرایه است و preprocessing با str() متن را به شکل repr پایتون به judge می‌داد (براکت و کوتیشن اضافه، newlineهای تبدیل‌شده به دو کاراکتر)؛ و claude-v1 از فهرست candidates جا افتاده بود، پس ۱۰ مدل به‌جای ۱۱ مدل ارزیابی می‌شد.

## ۲. کدام methodologyها و چرا

انتخاب بر اساس یک معیار روشن انجام شد: کدام روش‌ها پیش‌نیاز دادهٔ خود را با RouterBench برآورده می‌کنند و روی corpus ایستا قابل ارزیابی صادقانه هستند.

| # | Methodology | وضعیت | توضیح |
|---|---|---|---|
| **2** | `Matrix Factorization` | **پیاده شد** | **ماتریس sparse نمرات benchmark موجود است** |
| **5** | `Intent-Driven Anchoring` | **پیاده شد** | **vector anchor از خود corpus قابل ساخت است** |
| **10** | `KNN Historical` | **پیاده شد** | **تاریخچهٔ تصمیم‌های موفق همان training set است** |
| 4 | `Dynamic Thresholding` | **جزئی** | **به‌صورت مکانیزم sweep روی λ پیاده شد، نه router مستقل** |
| 1 | `Router-BERT` | **انجام نشد** | **نیاز به fine-tune روی GPU؛ داده و سخت‌افزارش موجود است** |
| 3 | `Entropy-Based Cascade` | **ناممکن** | **نیاز به logits مدل‌ها؛ RouterBench فقط نمرهٔ نهایی دارد** |
| 6 | `Token-Budget Routing` | **ناممکن** | **طول خروجی جدا ثبت نشده، فقط `total_cost`** |
| 7 | `Speculative Verification` | **ناممکن** | **نیاز به اجرای هم‌زمان دو مدل؛ خارج از دامنهٔ ارزیابی offline** |
| 8 | `Multi-Agent Meta-Routing` | **انجام نشد** | **نیاز به دیتاست segmented؛ `eval_name` با ۳۴ مقدار نامتوازن است** |
| 9 | `Reward-Model Guided` | **انجام نشد** | **reward model از پیش آموزش‌دیده در اختیار نداریم** |
| 11 | `Multi-Armed Bandit` | **عمداً نشد** | **نیاز به online reward loop؛ روی corpus ایستا فقط ادای exploration در می‌آید** |

**سه ردیف «ناممکن» تصمیم ما نبودند، محدودیت دیتاست‌اند؛ ارزیابی‌شان به داده‌ای فراتر از RouterBench نیاز دارد، مشخصاً logitهای مدل‌های self-hosted برای روش ۳.**

## ۳. معماری پیاده‌سازی

---

**هر سه router یک interface مشترک دارند: `fit(train)` و سپس `predict(prompts)` که دو ماتریس به ابعاد `n_prompts` × `11_models` برمی‌گرداند — کیفیت پیش‌بینی‌شده و هزینهٔ پیش‌بینی‌شده برای هر candidate.**

### ۳٫۱ تصمیم طراحی: selection مشترک است، نه داخل هر router

**هیچ‌کدام از routerها خودشان انتخاب نمی‌کنند. آن‌ها فقط پیش‌بینی می‌کنند و انتخاب در یک تابع مشترک انجام می‌شود:**

```
U = Q − λ · C   →   selection = argmax(U)
```

**دو دلیل پشت این تصمیم بود. اول اینکه مقایسه را منصفانه می‌کند — هر سه روش زیر یک objective یکسان قضاوت می‌شوند، پس تفاوت نتایج از کیفیت prediction می‌آید نه از تفاوت سیاست انتخاب. دوم اینکه sweep روی λ مستقیماً Pareto frontier را تولید می‌کند: λ صفر یعنی «کیفیت به هر قیمت» و λ بزرگ یعنی «ارزان‌ترین گزینه»؛ بیست مقدار جارو شد. همین sweep عملاً همان چیزی است که methodology شمارهٔ ۴ توصیف می‌کند، یعنی تنظیم پویای threshold بر اساس وزن هزینه — به همین دلیل آن را به‌عنوان مکانیزم داریم ولی به‌عنوان router مستقل ارزیابی نکردیم.**

### ۳٫۲ سه router

**KNN Historical (روش ۱۰). پرامپت‌های training با TF-IDF هش‌شده در ۲۰۴۸ بُعد نمایش داده می‌شوند. برای هر prompt جدید، ۳۲ همسایهٔ نزدیک با cosine similarity پیدا و میانگین وزن‌دار نمره و هزینهٔ آن‌ها به‌عنوان prediction برگردانده می‌شود. training واقعی ندارد؛ کل یادگیری همان lookup است.**

**Intent-Driven Anchoring (روش ۵).** روی training set یک spherical k-means با ۱۲۸ anchor اجرا می‌شود و برای هر anchor پروفایل میانگین یازده مدل از پیش محاسبه می‌گردد. تصمیم برای prompt جدید یعنی پیدا کردن نزدیک‌ترین anchor. تفاوت مهمش با KNN این است که هزینهٔ تصمیم مستقل از اندازهٔ corpus است — یک ضرب ماتریس در ۱۲۸ مرکز، نه جست‌وجو در ۲۹ هزار نمونه. خوشه‌های کوچک به سمت میانگین کل shrink می‌شوند، چون anchor با سه عضو پروفایل نیست، نویز است.

**Matrix Factorization (روش ۲).** اینجا از توصیف سند فاصله گرفتیم و دلیلش باید ثبت شود. factorization کلاسیک ماتریس prompt×model به تکرار سطر نیاز دارد تا embedding نهفتهٔ هر prompt یاد گرفته شود، ولی در RouterBench هر prompt یکتاست و هیچ سطری تکرار نمی‌شود. راه‌حل این بود که embedding را به تابعی خطی از featureها مقید کنیم و ماتریس نمره را با رتبهٔ محدود پیش‌بینی کنیم — عملاً `reduced-rank ridge regression` با حل بسته‌فرم. هزینه هم روی `log10` رگرس شد چون تقریباً خطی در طول prompt است و شرطی‌شدگی بهتری دارد.

### ۳٫۳ نکتهٔ روش‌شناختی

baseline قبلی هیچ train/test split نداشت. برای یک judge بدون state این بی‌ضرر است، ولی KNN و factorization بدون split عملاً دادهٔ training خودشان را به شما گزارش می‌دهند. تمام اعداد این گزارش روی ۷٬۲۹۶ نمونهٔ held-out محاسبه شده و مدل‌ها فقط روی ۲۹٬۱۸۵ نمونهٔ دیگر fit شده‌اند.

## ۴. نتایج

**۷٬۲۹۶ پرامپت held-out · سقف oracle: ۹۱٫۴۳٪ · بهترین سیاست ثابت: `always gpt-4` با ۷۸٫۱۱٪ و ۰٫۰۰۳۲۷۰ دلار**

| روش | Router | کیفیت | $/query | صرفه | کیفیت Δ | CI95 (paired) |
|---|---|---|---|---|---|---|
| **5** | anchor | 78.67% | 0.002751 | 16% | +0.57pp | [+0.22, +0.91] |
| 10 | knn | 77.81% | 0.002483 | 24% | −0.29pp | [−0.75, +0.16] |
| 2 | matrixfact | 76.55% | 0.002895 | 11% | −1.56pp | [−2.03, −1.10] |

هر router در نقطه‌ای مقایسه شده که هزینه‌اش از سیاست ثابت بیشتر نباشد، یعنی مقایسهٔ `cost-matched`. خواندن جدول:

- روش ۵ برنده است. تنها روشی که هم‌زمان کیفیت بالاتر و هزینهٔ کمتر دارد و بازهٔ اطمینانش صفر را رد می‌کند. این یک بهبود معنادار روی Pareto frontier است، نه نویز.
- روش ۱۰ در کیفیت با gpt-4 برابر است — بازه صفر را در بر می‌گیرد، پس تفاوت معنادار نیست — ولی با ۲۴٪ هزینهٔ کمتر. این هم برد است، منتها روی محور هزینه.
- روش ۲ معنادار بدتر است. بازه کاملاً زیر صفر است. دلیلش همان محدودیت ساختاری است: با promptهای یکتا، factorization چیزی برای فاکتور گرفتن ندارد.

روی Pareto frontier کامل، anchor تقریباً تمام نقاط non-dominated را می‌گیرد؛ برای نمونه در ۰٫۰۰۰۵۵۱ دلار به کیفیت ۷۴٫۲۰٪ می‌رسد، یعنی حدود ۸۳٪ ارزان‌تر از gpt-4 به بهای ۳٫۹ واحد درصد کیفیت. انتخاب نقطه روی این منحنی تصمیم محصولی است نه فنی، و همین که منحنی در دست است تصمیم را ممکن می‌کند.

### ۴٫۱ مقایسه با baseline موجود

روی همان ۱۰۰ پرامپتی که judge تصمیم گرفته بود، با fit شدن routerها روی ۳۶٬۳۸۱ پرامپت باقی‌مانده:

| روش | کیفیت | $/query | صرفه | زمان تصمیم |
|---|---|---|---|---|
| **LLM-as-judge (baseline)** | 61.25% | 0.000223 | – | ~2180 ms |
| matrixfact | 69.25% | 0.000203 | 9% | ~0.4 ms |
| knn | 67.75% | 0.000154 | 31% | ~0.4 ms |
| anchor | 64.75% | 0.000143 | 36% | ~0.4 ms |

**هر سه روش هم کیفیت بالاتر دارند، هم هزینهٔ کمتر، و حدود سه مرتبهٔ بزرگی سریع‌ترند. اختلاف زمان تصمیم مهم‌تر از چیزی است که در نگاه اول به‌نظر می‌رسد: بودجهٔ routing overhead در طراحی معماری $p99 \leq 25$ ms است. یک judge مبتنی بر LLM حدود نود برابر از این بودجه رد می‌شود؛ این سه روش حدود شصت برابر زیر آن می‌مانند.**

> **یک قید انصاف که باید در گزارش بماند: routerهای یادگیرنده روی ۲۹ هزار نمونهٔ برچسب‌دار fit شده‌اند و judge هیچ دادهٔ آموزشی نداشت. پس استدلال درست این نیست که «judge بد است»؛ استدلال درست این است که ما این داده را در اختیار داریم و روشی که از آن استفاده می‌کند با کسری از latency نتیجهٔ بهتری می‌دهد، بنابراین هزینهٔ یک فراخوانی LLM روی critical path توجیه ندارد.**

## ۵. اندازه‌گیری latency روی سخت‌افزار خودمان

**utility function سه جمله دارد: کیفیت، هزینه و latency. RouterBench دو تای اول را دارد و سومی را ندارد. نتیجه این بود که فیلد `predicted_total_latency_ms` در قرارداد event هیچ پشتوانه‌ای نداشت و عملاً روی یک فضای دوبعدی بهینه‌سازی می‌کردیم.**

**برای مدل‌هایی که روی یک کارت ۲۰ گیگابایتی قابل host بودند، TTFT و TPOT اندازه‌گیری شد. شصت نمونه به‌ازای هر مدل، روی شبکه‌ای درهم‌ریخته از پنج طول prompt و دو طول generation:**

| مدل | Quantization | TTFT (fit) | $R^2$ | TPOT | کل @ 256→512 |
|---|---|---|---|---|---|
| Mistral-7B | FP16 | 55.9 + 0.1525·tok | 0.90 | 43.6 ms | 11375 ms |
| WizardLM-13B | AWQ INT4 | 53.3 + 0.3100·tok | 1.00 | 23.1 ms | 6106 ms |

**نتیجهٔ جالب این اندازه‌گیری این است که مدل ۱۳ میلیاردی با INT4 در decode نزدیک دو برابر سریع‌تر از مدل ۷ میلیاردی با FP16 است، در حالی که در prefill دقیقاً برعکس و حدود دو برابر کندتر است. دلیلش این است که دو فاز به دو منبع متفاوت محدودند: decode به حجم بایت وزن‌ها محدود است و prefill به تعداد پارامتر. عملاً یعنی برای پاسخ‌های بلند، quantization سود بیشتری از کوچک کردن مدل می‌دهد — که پشتوانهٔ تجربی مستقیمی برای تصمیم معماری دربارهٔ لایهٔ محلی INT4 است.**

**اعتبار این اعداد با یک بررسی مستقل تأیید شد: هر دو در حدود ۹۰ درصد سقف نظری memory bandwidth این کارت قرار می‌گیرند (۹۲٪ و ۸۸٪). عددی که با فیزیک سخت‌افزار جور در بیاید اندازه‌گیری است، نه نویز.**

**نه مدل باقی‌مانده اندازه‌گیری نشدند و عمداً هیچ عددی برایشان حدس زده نشد؛ در پروفایل با برچسب `measured: false` ثبت شده‌اند. مدل‌های OpenAI و Anthropic به کلید API نیاز دارند و Mixtral و Llama-2-70B روی یک کارت ۲۰ گیگابایتی جا نمی‌شوند.**

---

تمام اعداد این گزارش روی held-out set محاسبه شده‌اند و با اجرای مجدد harness قابل بازتولیدند. فاصله‌های اطمینان با paired bootstrap و ۲۰ هزار draw محاسبه شده‌اند.